

LAB1 – INF1316 – Sistemas Operacionais – 2026.1

Criação de Processos

1. Identificação

Nome: Caio Faria Brito Martins Chaves

Matrícula: 2410162

Nome: Lucas Hufnagel Gromann de Araujo Goes

Matrícula: 2410845

2. Objetivo

O objetivo deste laboratório é implementar um programa em linguagem C que demonstre a criação de processos em sistemas operacionais Unix. Para isso, foi utilizada a chamada de sistema fork para criar três processos hierárquicos: pai, filho e neto.

O programa declara uma variável inteira n com valor inicial igual a 1. Cada processo executa um loop de 1000 iterações modificando essa variável de forma diferente e exibindo na tela o PID do processo através da função getpid.

Também é utilizada a função waitpid para garantir que o processo pai aguarde o término do filho e que o filho aguarde o término do neto.

3. Estrutura do Programa

O programa foi desenvolvido em C utilizando as bibliotecas:

- stdio.h
- unistd.h
- sys/wait.h
- stdlib.h

O programa possui a função main, onde são declaradas as variáveis n e status, realizadas as chamadas de fork() e executados os loops de cada processo.

4. Solução

O programa começa declarando a variável n e status com valor inicial igual a 1.

Em seguida é executado um fork() que cria o processo filho. O processo pai executa um loop de 1000 iterações somando 1 à variável n e exibindo seu PID e o valor atualizado da variável. Após terminar, o pai espera o filho finalizar usando waitpid().

Dentro do processo filho é executado outro fork() que cria o processo neto. O processo filho executa um loop de 1000 iterações somando 10 ao valor de n, exibindo seu PID e o valor da variável. Após o loop, ele aguarda o término do neto.

O processo neto executa um loop de 1000 iterações somando 100 ao valor de n e imprime seu PID e o valor da variável.

Como os processos executam em paralelo, as mensagens podem aparecer intercaladas no terminal.

5. Observações e Conclusões

Os valores impressos pelos processos são diferentes, pois cada processo possui sua própria cópia da variável n após a chamada de fork().

Como n começa com valor 1, os valores finais serão:

- Processo pai: 1001
- Processo filho: 10001
- Processo neto: 100001

Os resultados obtidos foram como o esperado.